

This call compares two rectangles and returns `True` if they overlap at all or `False` if they don't. We compare the ball rectangle with the target rectangle ⑤, and if they overlap, the program prints "Ball is touching the target" to the shell window.

The last change is where we draw both the target and the ball. The target is drawn first so that when the two overlap, the ball appears over the target ⑥.

When the program is run, if the rectangle of the ball overlaps the rectangle of the target, the message is written to the shell window. If you move the ball away from the target, the message stops being written out.

Dealing with Repeating Keys in Continuous Mode

The second way to handle keyboard interactions in pygame is to *poll* the keyboard. This involves asking pygame for a list representing which keys are currently down in every frame using the following call:

```
<aTuple> = pygame.key.get_pressed()
```

This call returns a tuple of 0s and 1s representing the state of each key: 0 if the key is up, 1 if the key is down. You can then use constants defined within pygame as an index into the returned tuple to see if a *particular* key is down. For example, the following lines can be used to determine the state of the A key:

```
keyPressedTuple = pygame.key.get_pressed()
# Now use a constant to get the appropriate element of the tuple
aIsDown = keyPressedTuple[pygame.K_a]
```

The full listing of constants representing all keys defined in pygame can be found at <https://www.pygame.org/docs/ref/key.html>.

The code in [Listing 5-5](#) shows how we can use this technique to move an image continuously rather than once per key press. In this version, we move the keyboard handling from section #7 to section #8. The rest of the code is identical to the previous version in [Listing 5-4](#).